SQL Trigger

A **SQL trigger** is a database object that automatically runs when a specified event occurs on a table, such as inserting, updating, or deleting a row.

The two trigger timings are `BEFORE` and `AFTER`.

Each can be combined with three operations:

Operation	Before	After
INSERT	BEFORE INSERT	AFTER INSERT
UPDATE	BEFORE UPDATE	AFTER UPDATE
DELETE	BEFORE DELETE	AFTER DELETE

That gives **six timing-and-operation combinations**.

Triggers are commonly used to record changes, enforce business rules, or maintain related data.

Example of AFTER UPDATE:

Suppose you have these tables:

```
CREATE TABLE courses (
    course_id INT PRIMARY KEY,
    course_name VARCHAR(100),
    tuition DECIMAL(10, 2)
);

CREATE TABLE tuition_history (
    history_id INT AUTO_INCREMENT PRIMARY KEY,
    course_id INT,
    old_tuition DECIMAL(10, 2),
    new_tuition DECIMAL(10, 2),
    changed_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP
);
```

It's relational tables are:
Courses(course_id, course_name, tuition)
Tuition_history(history_id, course_id, old_tuition, new_tuition, changed_at)

Create a trigger that runs after a course is updated (this syntax is specifically for MySQL):

```
DELIMITER //

CREATE TRIGGER record_tuition_change
AFTER UPDATE ON courses
FOR EACH ROW
BEGIN
    IF NOT (OLD.tuition <=> NEW.tuition) THEN
        INSERT INTO tuition_history (
            course_id,
            old_tuition,
            new_tuition
        )
        VALUES (
            NEW.course_id,
            OLD.tuition,
            NEW.tuition
        );
    END IF;
END//

DELIMITER ;
```

Here is how it works:

- **`AFTER UPDATE`** runs the trigger after a row is updated.
- **`FOR EACH ROW`** runs it for every affected row.
- **`BEGIN/END`** groups the SQL statements that make up the trigger's body, marking where its actions start and finish**.**
- `//` tells the MySQL client that the entire `CREATE TRIGGER` statement is complete and ready to execute.
- **`OLD.tuition`** is the tuition value stored in a row before an update**.** Inside a trigger, the database **automatically** provides `OLD` and **`NEW`** so you can compare the row before and after the change.
- **`NEW.tuition`** contains the tuition after the update.
- **`<=>`** is MySQL's NULL-safe equality operator. The condition checks whether the value changed, including changes involving `NULL`.
- write a space between **DELIMITER** and **;**. The semicolon in DELIMITER ; is the **new delimiter value**, not punctuation ending the command.

MySQL requires **FOR EACH ROW** in trigger definitions because it supports row-level triggers only. Even when you insert just one row updated, you must include it.

For example:

```
INSERT INTO courses
VALUES (1, 'EKG', 949.00);

UPDATE courses
SET tuition = 999.00
WHERE course_id = 1;
```

The trigger automatically adds a history record showing the change from **$949 to $999**. View it with:

```
SELECT * FROM tuition_history;
```

Triggers can run **before** or **after** an `INSERT`, `UPDATE`, or `DELETE`. Use them carefully: because they run automatically, they can make database behavior harder to trace and add work to each affected operation.

The exact syntax varies between different DBMS: MySQL, SQL Server, PostgreSQL, and other databases.

DELIMITER

In MySQL tools such as the command-line client and MySQL Workbench, `DELIMITER` changes the marker that tells the tool **where a complete SQL statement ends**. It is a client command, rather than SQL executed by the server.

Normally, that marker is a semicolon:

```
SELECT * FROM courses;
```

When creating a trigger, its body can contain several statements ending in semicolons. To send the entire trigger definition together, temporarily use a different ending marker, such as `//`:

CHECK Constraint example:

A **`CHECK` constraint** is usually the simpler choice for preventing negative tuition.

You can define it when creating the table:

```
CREATE TABLE courses (
    course_id INT PRIMARY KEY,
```

```
    course_name VARCHAR(100),
    tuition DECIMAL(10, 2) NOT NULL,
    CONSTRAINT tuition_nonnegative CHECK (tuition >= 0)
);
```

- **`CHECK (tuition >= 0)`** allows zero or positive tuition.
- **`NOT NULL`** requires a tuition value.

The constraint applies to **both inserts and updates**, so a negative value is rejected in either case.

Example of AFTER INSERT

Another example is **automatically recording payments against a student's outstanding balance**.

When a payment is inserted, the trigger subtracts it from the student's balance—all within the same transaction.

```
CREATE TABLE student_accounts (
    student_id INT PRIMARY KEY,
    student_name VARCHAR(100) NOT NULL,
    balance DECIMAL(10, 2) NOT NULL
);

CREATE TABLE payments (
    payment_id INT AUTO_INCREMENT PRIMARY KEY,
    student_id INT NOT NULL,
    amount DECIMAL(10, 2) NOT NULL CHECK (amount > 0),
    paid_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
    FOREIGN KEY (student_id)
        REFERENCES student_accounts (student_id)
);

DELIMITER //

CREATE TRIGGER apply_student_payment
AFTER INSERT ON payments
FOR EACH ROW
BEGIN
    UPDATE student_accounts
    SET balance = balance - NEW.amount
    WHERE student_id = NEW.student_id;
END//

DELIMITER ;

INSERT INTO student_accounts (
    student_id, student_name, balance
)
VALUES (1, 'Maria Garcia', 949.00);
```

```
INSERT INTO payments (student_id, amount)
VALUES (1, 200.00);

SELECT * FROM student_accounts;
```

Result:

student_id	student_name	balance
1	Maria Garcia	749.00

The trigger connects two operations:

- **`AFTER INSERT ON payments`** runs whenever a payment is recorded.
- **`NEW.amount`** is the payment amount.
- **`NEW.student_id`** identifies whose balance to reduce.
- **`UPDATE student_accounts`** automatically applies the payment.

A `CHECK` constraint can validate a payment amount, but it cannot perform this update to another table. This example assumes payment records are kept unchanged.

Example of BEFORE UPDATE

For example, this trigger puts a cap on the max amount a tuition can be updated.

```
DELIMITER //

CREATE TRIGGER cap_course_tuition
BEFORE UPDATE ON courses
FOR EACH ROW
BEGIN
    IF NEW.tuition > 2000.00 THEN
        SET NEW.tuition = 2000.00;
    END IF;
END//

DELIMITER ;
```

- `BEFORE UPDATE` runs before the tuition is saved.
- `NEW.tuition` contains the tuition being inserted, caps it to 2000 if greater than 2000.